

Classes e Objetos

Introdução à Programação Orientada a Objetos

Ana Júlia Lima



Desenvolvido para
Young 1

Ctrl Play Santa Mônica
Vila Velha, Brasil
Fevereiro 2026

Sumário

1	Programação Orientada a Objetos	3
2	Antes de entender POO: o que são classes?	3
3	Classe	3
3.1	O que é uma classe?	3
3.2	Como criar uma classe?	4
3.3	Exemplo simples	4
4	Atributos	4
4.1	O que são atributos?	4
4.2	Qual a utilidade dos atributos?	4
4.3	Como criar atributos?	4
5	Métodos	5
5.1	O que são métodos?	5
5.2	Qual a utilidade dos métodos?	5
5.3	Como criar métodos?	5
6	Objetos	5
6.1	O que é um objeto?	5
6.2	Como criar objetos?	5
6.3	Usando atributos e métodos do objeto	6
6.4	Função type()	6
7	Método inicializador __init__()	6
7.1	O que é o __init__()?	6
8	O que é self?	6
9	Getters e Setters	7
9.1	O que são getters?	7
9.2	O que são setters?	7
9.3	Qual a diferença entre getter e setter?	7
9.4	Para que e quando usar?	7
10	Métodos especiais	8
10.1	__init__()	8
10.2	__str__()	8
10.3	__len__()	9
10.4	__del__()	9
11	Projeto Criativo — Classe Personagem Mágico	9
11.1	Código completo	10
11.2	Usando a classe	10
11.3	Exemplo de saída	11
12	Lista de Exercícios — Classes e Objetos	12
13	Gabarito — Classes e Objetos	14

14 Mini Projeto Final — Biblioteca Mágica	18
14.1 Código completo do projeto	18
14.2 Exemplo de saída	19
14.3 O que esse projeto usa?	20
14.4 Desafios extras	20

1 Programação Orientada a Objetos

Programação Orientada a Objetos, também chamada de **POO**, é uma forma de organizar o código usando **classes** e **objetos**.

Em vez de escrever tudo solto no programa, criamos modelos que representam coisas do mundo real ou do próprio sistema.

Por exemplo:

- um aluno
- um personagem de jogo
- uma conta bancária
- um produto
- um carro

A ideia principal da POO é juntar:

- **dados** → características do objeto
- **ações** → comportamentos do objeto

Em Python, esses dados são chamados de **atributos**, e essas ações são chamadas de **métodos**.

2 Antes de entender POO: o que são classes?

Antes de criar objetos, precisamos entender o que é uma classe.

Uma **classe** funciona como um molde.

Ela define como um objeto será criado.

Pense em uma classe como uma receita de bolo:

- a receita explica como fazer o bolo
- cada bolo feito com a receita é um objeto

Ou seja:

- **classe** → modelo
- **objeto** → algo criado a partir desse modelo

3 Classe

3.1 O que é uma classe?

Classe é uma estrutura usada para criar objetos.

Ela pode guardar:

- atributos
- métodos

Exemplo de ideia:

- Classe: **Pessoa**
- Objetos: Ana, Carlos, Marina

3.2 Como criar uma classe?

Para criar uma classe usamos a palavra-chave `class`.

```
class Pessoa:
    pass
```

O comando `pass` é usado quando ainda não queremos escrever nada dentro da classe.

3.3 Exemplo simples

```
class Pessoa:
    nome = "Ana"
    idade = 18
```

Nesse exemplo, criamos uma classe chamada `Pessoa` com dois atributos:

- `nome`
- `idade`

4 Atributos

4.1 O que são atributos?

Atributos são as características de um objeto.
Eles representam informações que o objeto possui.
Por exemplo, uma pessoa pode ter:

- `nome`
- `idade`
- `cidade`

Um personagem de jogo pode ter:

- `nome`
- `vida`
- `força`
- `nível`

4.2 Qual a utilidade dos atributos?

Atributos servem para guardar dados dentro dos objetos.
Assim, cada objeto pode ter suas próprias informações.

4.3 Como criar atributos?

Uma forma mais correta e comum de criar atributos é usando o método `__init__()`.

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
```

Aqui, cada pessoa criada poderá ter um nome e uma idade diferentes.

5 Métodos

5.1 O que são métodos?

Métodos são funções que pertencem a uma classe.

Eles representam ações ou comportamentos que um objeto pode realizar.

Exemplo:

- uma pessoa pode se apresentar
- um carro pode acelerar
- um personagem pode atacar

5.2 Qual a utilidade dos métodos?

Métodos servem para fazer o objeto executar ações.

5.3 Como criar métodos?

Criamos métodos como funções dentro da classe.

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def apresentar(self):
        print("Olá, meu nome é", self.nome)
```

6 Objetos

6.1 O que é um objeto?

Objeto é algo criado a partir de uma classe.

Se a classe é o molde, o objeto é a coisa pronta.

Exemplo:

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
```

```
pessoa1 = Pessoa("Ana", 18)
pessoa2 = Pessoa("Carlos", 20)
```

Nesse exemplo:

- Pessoa é a classe
- pessoa1 é um objeto
- pessoa2 é outro objeto

6.2 Como criar objetos?

Para criar um objeto, usamos o nome da classe como se fosse uma função.

```
pessoa1 = Pessoa("Ana", 18)
```

6.3 Usando atributos e métodos do objeto

O objeto pode usar os atributos e métodos da classe.
Para isso, usamos um ponto depois do nome do objeto.

```
print(pessoa1.nome)
print(pessoa1.idade)

pessoa1.apresentar()
```

6.4 Função type()

A função type() mostra o tipo de um dado ou objeto.

```
print(type(pessoa1))
# Saída:
# <class '__main__.Pessoa'>
```

Isso mostra que pessoa1 pertence à classe Pessoa.

7 Método inicializador __init__()

7.1 O que é o __init__()?

O método __init__() é chamado automaticamente quando um objeto é criado.
Ele serve para inicializar os atributos do objeto.

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
```

Quando fazemos:

```
pessoa = Pessoa("Ana", 18)
```

O Python chama o __init__() automaticamente.

8 O que é self?

O self representa o próprio objeto.
Ele é usado para acessar os atributos e métodos daquele objeto.

```
class Pessoa:
    def __init__(self, nome):
        self.nome = nome

    def apresentar(self):
        print("Meu nome é", self.nome)
```

Quando usamos:

```
pessoa = Pessoa("Ana")
pessoa.apresentar()
```

```
# Saída:
# Meu nome é Ana
```

O self.nome se refere ao nome daquele objeto específico.

9 Getters e Setters

9.1 O que são getters?

Getters são métodos usados para **pegar** ou **acessar** um valor de forma controlada.

```
class Pessoa:
    def __init__(self, nome):
        self.nome = nome

    def get_nome(self):
        return self.nome
```

Uso:

```
pessoa = Pessoa("Ana")
print(pessoa.get_nome())
```

```
# Saída:
# Ana
```

9.2 O que são setters?

Setters são métodos usados para **alterar** um valor de forma controlada.

```
class Pessoa:
    def __init__(self, nome):
        self.nome = nome

    def set_nome(self, novo_nome):
        self.nome = novo_nome
```

Uso:

```
pessoa = Pessoa("Ana")
pessoa.set_nome("Marina")
```

```
print(pessoa.nome)
```

```
# Saída:
# Marina
```

9.3 Qual a diferença entre getter e setter?

Característica	Getter	Setter
Serve para acessar valor	Sim	Não
Serve para alterar valor	Não	Sim
Pode ter validação	Sim	Sim
Retorna informação	Sim	Geralmente não

9.4 Para que e quando usar?

Getters e setters são úteis quando queremos controlar melhor o acesso aos dados. Por exemplo, não queremos que uma idade negativa seja aceita.

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
```



```

def set_idade(self, nova_idade):
    if nova_idade >= 0:
        self.idade = nova_idade
    else:
        print("Idade inválida.")

def get_idade(self):
    return self.idade

```

Uso:

```

pessoa = Pessoa("Ana", 18)

pessoa.set_idade(-5)
# Saída:
# Idade inválida.

print(pessoa.get_idade())
# Saída:
# 18

```

10 Métodos especiais

Métodos especiais são métodos que possuem dois underlines antes e depois do nome. Eles também são chamados de **dunder methods**.

Exemplos:

- `__init__()`
- `__str__()`
- `__len__()`
- `__del__()`

10.1 `__init__()`

Usado para inicializar o objeto.

```

class Livro:
    def __init__(self, titulo):
        self.titulo = titulo

```

10.2 `__str__()`

Define como o objeto aparece quando usamos `print()`.

```

class Livro:
    def __init__(self, titulo):
        self.titulo = titulo

    def __str__(self):
        return "Livro: " + self.titulo

```

```

livro = Livro("Python Básico")

```

```

print(livro)
# Saída:
# Livro: Python Básico

```

10.3 `__len__()`

Define o que acontece quando usamos `len()` no objeto.

```
class Mochila:
    def __init__(self):
        self.itens = []

    def __len__(self):
        return len(self.itens)

mochila = Mochila()

print(len(mochila))
# Saída:
# 0
```

10.4 `__del__()`

É chamado quando o objeto é destruído.

```
class Personagem:
    def __init__(self, nome):
        self.nome = nome

    def __del__(self):
        print(self.nome, "foi removido da memória.")

heroi = Personagem("Luna")
del heroi

# Saída:
# Luna foi removido da memória.
```

11 Projeto Criativo — Classe Personagem Mágico

Agora vamos criar uma classe mais completa e criativa, usando vários conceitos da apostila.

A ideia será criar um personagem mágico de jogo.

Ele terá:

- atributos
- métodos
- método `__init__()`
- `self`
- getters
- setters
- métodos especiais

11.1 Código completo

```
class PersonagemMagico:
    def __init__(self, nome, elemento, vida, mana):
        self.nome = nome
        self.elemento = elemento
        self.vida = vida
        self.mana = mana
        self.magias = []

    def __str__(self):
        return f"[{self.nome}, mago(a) do elemento {self.elemento}]"

    def __len__(self):
        return len(self.magias)

    def __del__(self):
        print(self.nome, "desapareceu em uma nuvem de magia.")

    def get_vida(self):
        return self.vida

    def set_vida(self, nova_vida):
        if nova_vida >= 0:
            self.vida = nova_vida
        else:
            print("A vida não pode ser negativa.")

    def get_mana(self):
        return self.mana

    def set_mana(self, nova_mana):
        if nova_mana >= 0:
            self.mana = nova_mana
        else:
            print("A mana não pode ser negativa.")

    def aprender_magia(self, magia):
        self.magias.append(magia)
        print(self.nome, "aprendeu a magia:", magia)

    def mostrar_magias(self):
        if len(self.magias) == 0:
            print(self.nome, "ainda não aprendeu magias.")
        else:
            print("Magias de", self.nome)

            for magia in self.magias:
                print("-", magia)

    def atacar(self):
        if self.mana >= 10:
            self.mana -= 10
            print(self.nome, "lançou um ataque de", self.elemento)
        else:
            print(self.nome, "não tem mana suficiente.")
```

11.2 Usando a classe

```
personagem = PersonagemMagico("Luna", "Lua", 100, 30)
```

```

print(personagem)
# Saída:
# Luna, mago(a) do elemento Lua

personagem.aprender_magia("Raio Lunar")
personagem.aprender_magia("Escudo Estelar")

personagem.mostrar_magias()

print("Quantidade de magias:", len(personagem))

personagem.atacar()
print("Mana atual:", personagem.get_mana())

personagem.set_vida(80)
print("Vida atual:", personagem.get_vida())

```

11.3 Exemplo de saída

```

Luna, mago(a) do elemento Lua
Luna aprendeu a magia: Raio Lunar
Luna aprendeu a magia: Escudo Estelar
Magias de Luna
- Raio Lunar
- Escudo Estelar
Quantidade de magias: 2
Luna lançou um ataque de Lua
Mana atual: 20
Vida atual: 80

```

Resumo Final

- POO organiza programas usando classes e objetos.
- Classe é o molde.
- Objeto é criado a partir da classe.
- Atributos guardam características.
- Métodos representam ações.
- `__init__()` inicializa o objeto.
- `self` representa o próprio objeto.
- Getters acessam valores.
- Setters alteram valores com controle.
- Métodos especiais personalizam comportamentos do objeto.

12 Lista de Exercícios — Classes e Objetos

Nível 1 — Criando classes e objetos

1. Crie uma classe chamada `Pessoa` com os atributos:

- nome
- idade

Depois, crie um objeto e mostre seus atributos.

2. Crie uma classe chamada `Carro` com os atributos:

- marca
- modelo
- velocidade

Crie um objeto e imprima os dados.

3. Crie uma classe chamada `Animal` com um método chamado `emitir_som()`.

Esse método deve mostrar:

```
"O animal fez um som."
```

4. Use a função `type()` em um objeto criado por você.

Nível 2 — Métodos, `self` e `__init__()`

1. Crie uma classe chamada `Aluno` usando `__init__()`.

A classe deve receber:

- nome
- nota

2. Crie um método chamado `mostrar_dados()` que mostre os dados do aluno.

3. Crie uma classe chamada `ContaBancaria` com:

- titular
- saldo

Crie um método chamado `depositar()`.

4. Crie um método chamado `sacar()` que diminua o saldo apenas se houver dinheiro suficiente.

Nível 3 — Getters, setters e métodos especiais

1. Crie uma classe chamada `Produto` com:

- nome
- preco

Crie getters e setters para o preço.

2. Faça o setter impedir preços negativos.

3. Crie uma classe chamada `Playlist` com:

- nome

- lista de músicas

Implemente:

- `__str__()`
- `__len__()`

4. Crie uma classe livre usando:

- atributos
- métodos
- `__init__()`
- getters
- setters
- pelo menos um método especial

13 Gabarito — Classes e Objetos

Nível 1

1.

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
```

```
pessoa = Pessoa("Ana", 18)
```

```
print(pessoa.nome)
print(pessoa.idade)
```

```
# Saída:
# Ana
# 18
```

2.

```
class Carro:
    def __init__(self, marca, modelo, velocidade):
        self.marca = marca
        self.modelo = modelo
        self.velocidade = velocidade
```

```
carro = Carro("Toyota", "Corolla", 120)
```

```
print(carro.marca)
print(carro.modelo)
print(carro.velocidade)
```

3.

```
class Animal:

    def emitir_som(self):
        print("O animal fez um som.")
```

```
animal = Animal()
```

```
animal.emitir_som()
```

4.

```
class Pessoa:
    pass
```

```
objeto = Pessoa()
```

```
print(type(objeto))
```

```
# Saída:
# <class '__main__.Pessoa'>
```

Nível 2

1.

```
class Aluno:

    def __init__(self, nome, nota):
        self.nome = nome
        self.nota = nota
```

2.

```
class Aluno:

    def __init__(self, nome, nota):
        self.nome = nome
        self.nota = nota

    def mostrar_dados(self):
        print("Nome:", self.nome)
        print("Nota:", self.nota)
```

```
aluno = Aluno("Carlos", 9)
```

```
aluno.mostrar_dados()
```

3.

```
class ContaBancaria:

    def __init__(self, titular, saldo):
        self.titular = titular
        self.saldo = saldo

    def depositar(self, valor):
        self.saldo += valor
```

```
conta = ContaBancaria("Ana", 100)
```

```
conta.depositar(50)
```

```
print(conta.saldo)
```

```
# Saída:
# 150
```

4.

```
class ContaBancaria:

    def __init__(self, titular, saldo):
        self.titular = titular
        self.saldo = saldo

    def sacar(self, valor):

        if valor <= self.saldo:
            self.saldo -= valor
            print("Saque realizado.")
        else:
            print("Saldo insuficiente.")
```

```
conta = ContaBancaria("Ana", 100)
```

```
conta.sacar(30)
```



```
print(conta.saldo)
```

```
# Saída:  
# Saque realizado.  
# 70
```

Nível 3

1 e 2.

```
class Produto:  
  
    def __init__(self, nome, preco):  
        self.nome = nome  
        self.preco = preco  
  
    def get_preco(self):  
        return self.preco  
  
    def set_preco(self, novo_preco):  
  
        if novo_preco >= 0:  
            self.preco = novo_preco  
        else:  
            print("Preço inválido.")
```

```
produto = Produto("Notebook", 3000)
```

```
produto.set_preco(3500)
```

```
print(produto.get_preco())
```

```
# Saída:  
# 3500
```

3.

```
class Playlist:  
  
    def __init__(self, nome):  
        self.nome = nome  
        self.musicas = []  
  
    def adicionar_musica(self, musica):  
        self.musicas.append(musica)  
  
    def __str__(self):  
        return f"Playlist: {self.nome}"  
  
    def __len__(self):  
        return len(self.musicas)
```

```
playlist = Playlist("Favoritas")
```

```
playlist.adicionar_musica("Musica 1")
```

```
playlist.adicionar_musica("Musica 2")
```

```
print(playlist)  
print(len(playlist))
```

```
# Saída:  
# Playlist: Favoritas  
# 2
```

4. Exemplo livre

```
class Pet:  
  
    def __init__(self, nome, energia):  
        self.nome = nome  
        self.energia = energia  
  
    def brincar(self):  
        self.energia -= 10  
        print(self.nome, "brincou!")  
  
    def get_energia(self):  
        return self.energia  
  
    def set_energia(self, valor):  
  
        if valor >= 0:  
            self.energia = valor  
  
    def __str__(self):  
        return f"Pet: {self.nome}"  
  
pet = Pet("Bolt", 100)  
  
print(pet)  
  
pet.brincar()  
  
print(pet.get_energia())
```

14 Mini Projeto Final — Biblioteca Mágica

Neste mini projeto, vamos criar uma **biblioteca mágica** usando Programação Orientada a Objetos. A ideia é criar livros mágicos que possuem:

- título
- autor
- número de páginas
- nível de magia
- status de empréstimo

Cada livro poderá:

- ser emprestado
- ser devolvido
- aumentar seu nível de magia
- mostrar suas informações

14.1 Código completo do projeto

```
class LivroMagico:

    def __init__(self, titulo, autor, paginas, nivel_magia):
        self.titulo = titulo
        self.autor = autor
        self.paginas = paginas
        self.nivel_magia = nivel_magia
        self.emprestado = False

    def __str__(self):
        return f"{self.titulo}, escrito por {self.autor}"

    def __len__(self):
        return self.paginas

    def get_nivel_magia(self):
        return self.nivel_magia

    def set_nivel_magia(self, novo_nivel):

        if novo_nivel >= 0:
            self.nivel_magia = novo_nivel
        else:
            print("0 nível de magia não pode ser negativo.")

    def emprestar(self):

        if self.emprestado == False:
            self.emprestado = True
            print("0 livro foi emprestado com sucesso.")
        else:
            print("Esse livro já está emprestado.")

    def devolver(self):
```

```

        if self.emprestado == True:
            self.emprestado = False
            print("O livro foi devolvido.")
        else:
            print("Esse livro já estava na biblioteca.")

    def encantar(self):
        self.nivel_magia += 1
        print("O livro brilhou e ganhou mais poder mágico!")

    def mostrar_informacoes(self):
        print("\n--- LIVRO MÁGICO ---")
        print("Título:", self.titulo)
        print("Autor:", self.autor)
        print("Páginas:", self.paginas)
        print("Nível de magia:", self.nivel_magia)

        if self.emprestado:
            print("Status: emprestado")
        else:
            print("Status: disponível")

livro = LivroMagico("O Códice das Estrelas", "Luna Silver", 280, 5)

print(livro)
print("Quantidade de páginas:", len(livro))

livro.mostrar_informacoes()

livro.emprestar()
livro.encantar()
livro.devolver()

livro.set_nivel_magia(10)

print("Novo nível de magia:", livro.get_nivel_magia())

livro.mostrar_informacoes()

```

14.2 Exemplo de saída

O Códice das Estrelas, escrito por Luna Silver
Quantidade de páginas: 280

```

--- LIVRO MÁGICO ---
Título: O Códice das Estrelas
Autor: Luna Silver
Páginas: 280
Nível de magia: 5
Status: disponível
O livro foi emprestado com sucesso.
O livro brilhou e ganhou mais poder mágico!
O livro foi devolvido.
Novo nível de magia: 10

--- LIVRO MÁGICO ---
Título: O Códice das Estrelas
Autor: Luna Silver
Páginas: 280

```

Nível de magia: 10
Status: disponível

14.3 O que esse projeto usa?

- `class` para criar a classe
- `__init__()` para inicializar o objeto
- atributos como título, autor, páginas e nível de magia
- métodos como `emprestar()`, `devolver()` e `encantar()`
- getter e setter para controlar o nível de magia
- `__str__()` para mostrar o livro de forma bonita
- `__len__()` para representar a quantidade de páginas

14.4 Desafios extras

Tente melhorar a biblioteca mágica:

- criar mais de um livro mágico
- guardar vários livros em uma lista
- criar uma classe `BibliotecaMagica`
- permitir buscar livros pelo título
- impedir livros com quantidade de páginas negativa
- adicionar um método para comparar qual livro é mais poderoso